Finance Data Processing and Access Control Backend

Overview

This project is a backend system for a finance dashboard where different users interact with financial records based on their role. The backend provides APIs for user management, financial record management, dashboard analytics, and role-based access control.

The assignment required support for:

- user and role management
- financial records CRUD
- filtering
- dashboard summary APIs
- backend access control
- validation and error handling
- data persistence

This implementation also includes optional enhancements such as JWT authentication, Swagger API documentation, and basic automated tests, which are explicitly listed as appreciated additions in the assignment brief.

Objective

The goal of this project is to demonstrate backend design, API structuring, business logic implementation, role-based behavior, and data handling in a clean and maintainable way. The assignment emphasizes correctness, clarity, maintainability, and logical organization over unnecessary complexity.

Tech Stack

- **Language:** Python
- **Framework:** FastAPI
- **Database:** SQLite
- **ORM:** SQLAlchemy
- **Validation:** Pydantic
- **Authentication:** JWT-based token authentication
- **Testing:** Pytest
- **API Docs:** Swagger UI via FastAPI

Why These Choices

FastAPI

FastAPI was chosen because it makes it easy to build clean REST APIs, provides automatic Swagger/OpenAPI documentation, and has built-in validation through Pydantic.

SQLite

SQLite was used because it is lightweight, easy to set up, and suitable for an assignment project. It avoids the need for external database installation while still providing persistent storage. The assignment specifically allows SQLite for simplicity.

SQLAlchemy

SQLAlchemy provides a clean ORM layer for working with database tables and helps keep models and queries maintainable.

JWT Authentication

JWT token authentication was implemented as an optional enhancement because the assignment allows token/session authentication as an additional improvement.

Features Implemented

This project includes the following features:

- User and role management
- Active/inactive user status handling
- Financial records CRUD
- Record filtering by date, category, and type
- Dashboard summary APIs
- Role-based access control
- Input validation and structured error handling
- SQLite database persistence
- JWT authentication
- Swagger documentation
- Seed endpoint for demo users
- Basic automated tests

These map directly to the assignment requirements and optional enhancements.

Role Model

The project uses three main roles:

Viewer

- Can view dashboard data
- Can read allowed data
- Cannot create, update, or delete records
- Cannot manage users

Analyst

- Can view records
- Can access dashboard insights and summaries
- Cannot manage users
- Cannot perform full administrative actions

Admin

- Full access
- Can create, update, and manage records
- Can create and manage users
- Can update user status

This role model follows the assignment's suggested role examples.

Project Structure

```
finance_backend_assignment/
 ├── app/
|   ├── __init__.py
|   ├── auth.py
|   ├── db.py
|   ├── dependencies.py
|   ├── main.py
|   ├── models.py
|   ├── schemas.py
|   ├── seed.py
|   └── routes/
|       ├── dashboard.py
|       ├── records.py
|       └── users.py
 ├── tests/
|   └── test_app.py
 ├── finance_dashboard.db
```

```
├── requirements.txt
└── README.md
```

Structure Explanation

- main.py — application entry point
- db.py — database engine, session, and base setup
- models.py — SQLAlchemy models
- schemas.py — Pydantic request/response schemas
- auth.py — JWT token handling and password verification logic
- dependencies.py — shared dependencies like current user, role checking
- seed.py — demo user creation logic
- routes/users.py — user management APIs
- routes/records.py — financial record APIs
- routes/dashboard.py — analytics/summary APIs
- tests/test_app.py — basic tests

This modular structure supports separation of concerns, which is one of the assignment's evaluation points.

Setup Instructions

1. Extract the project

Unzip the project folder.

2. Open in VS Code

Open the inner project folder in VS Code.

3. Create virtual environment

```
python -m venv venv
```

4. Activate virtual environment

PowerShell

```
.\venv\Scripts\Activate.ps1
```

CMD

```
venv\Scripts\activate
```

5. Install dependencies

```
pip install -r requirements.txt
```

6. Run the server

uvicorn app.main:app --reload

If needed:

python -m uvicorn app.main:app --reload

7. Open API docs

http://127.0.0.1:8000/docs

Root Endpoint

GET /

Health check endpoint.

Sample response

```
{
  "message": "Finance backend is running"
}
```

Demo User Setup

Before login, call:

POST /seed

This creates demo users if they do not already exist.

Sample response

```
{
  "message": "Seed completed",
  "users_created": 0
}
```

If users already exist, users_created may be 0.

Demo Credentials

Use these sample users for testing:

Admin

- Email: admin@example.com
- Password: Admin@123

Analyst

- Email: analyst@example.com
- Password: Analyst@123

Viewer

- Email: viewer@example.com
- Password: Viewer@123

Inactive user

- Email: inactive@example.com
- Password: Inactive@123

Authentication Flow

Login Endpoint

POST /auth/login

Request body

```
{
  "email": "admin@example.com",
  "password": "Admin@123"
}
```

Sample response

```
{
  "access_token": "your_jwt_token_here",
  "token_type": "bearer"
}
```

Using the token in Swagger

Click **Authorize** in Swagger and paste:

Bearer your_jwt_token_here

After authorization, protected endpoints can be used.

API Documentation

User APIs

1. Create User

POST /users

Used to create a new user.

Request body

```
{
  "name": "Test User",
  "email": "testuser@example.com",
```

```
  "password": "Test@123",
  "role": "viewer"
}
```

Sample response

```
{
  "id": 1,
  "name": "Test User",
  "email": "testuser@example.com",
  "role": "viewer",
  "is_active": true,
  "created_at": "2026-04-05T05:27:38.762Z"
}
```

Access

- Admin only

---

2. Get Users

GET /users

Returns list of users.

Access

- Admin only

---

3. Update User Status

PATCH /users/{user_id}/status

Used to activate or deactivate a user.

Sample request body

```
{
  "is_active": false
}
```

Access

- Admin only

Financial Record APIs

1. Create Record

POST /records

Creates a financial record.

Request body

```
{
  "amount": 1200.5,
  "type": "expense",
  "category": "software",
  "date": "2026-04-01",
  "notes": "Annual tool subscription"
}
```

Access

- Admin

---

2. Get Records

GET /records

Returns all records.

Access

- Admin, Analyst, Viewer depending on implemented access policy
- Read access is available for permitted roles

---

3. Filter Records

GET /records?type=expense&category=software&date=2026-04-01

Used to filter records by:

- type
- category
- date

This directly aligns with the assignment requirement for filtering by date, category, or type.

---

4. Update Record

PUT /records/{record_id}

Updates an existing record.

Sample request body

```
{
  "amount": 1400,
  "type": "expense",
  "category": "software",
```

```
  "date": "2026-04-01",
  "notes": "Updated annual subscription"
}
```


Access

- Admin only

---

5. Delete Record

DELETE /records/{record_id}

Deletes an existing record.

Access

- Admin only

Dashboard APIs

The assignment specifically asks for summary-level APIs such as total income, total expense, net balance, category-wise totals, recent activity, and trends.

1. Dashboard Summary

GET /dashboard/summary

Returns:

- total income
- total expenses
- net balance

Sample response


```
{
  "total_income": 5000,
  "total_expenses": 1200.5,
  "net_balance": 3799.5
}
```


---

2. Category Totals

GET /dashboard/category-totals

Returns totals grouped by category.

Sample response


```
[
  {
```

```
    "category": "software",
    "total": 1200.5
  },
  {
    "category": "salary",
    "total": 5000
  }
]
```


---

3. Recent Activity

GET /dashboard/recent-activity

Returns most recent financial records.

Sample response


```
[
  {
    "id": 1,
    "amount": 1200.5,
    "type": "expense",
    "category": "software",
    "date": "2026-04-01",
    "notes": "Annual tool subscription"
  }
]
```


---

4. Monthly Trends

GET /dashboard/monthly-trends

Returns monthly grouped trend data for dashboard use.

Sample response


```
[
  {
    "month": "2026-04",
    "income": 5000,
    "expense": 1200.5
  }
]
```

Access Control

The assignment requires clear backend-level enforcement of which user can perform which action.

This project enforces access control using JWT authentication plus role-based permission checks.

Access Matrix

Viewer

- Read-only access to allowed endpoints
- No create/update/delete
- No user management

Analyst

- Read records
- Access summaries and analytics
- No user creation or management
- No full admin actions

Admin

- Full user management
- Full record management
- Full dashboard access

Inactive User

- Login may work depending on implementation flow, but protected actions are blocked
- Inactive status is enforced at backend level

Validation and Error Handling

The assignment expects:

- input validation
- useful error responses
- proper status codes
- invalid operation protection

This project handles validation using Pydantic schemas and FastAPI validation.

Examples

Missing fields

If a required field is missing, FastAPI returns:

```
{
  "detail": [
    {
      "type": "missing",
```

```
    "loc": ["body", "name"],
    "msg": "Field required"
  }
 ]
}
```


Invalid login

Returns:

- 401 Unauthorized

Forbidden action by role

Returns:

- 403 Forbidden

Not found resource

Returns:

- 404 Not Found

Invalid request body

Returns:

- 422 Unprocessable Entity

Database Design

User Model

Typical fields:

- id
- name
- email
- hashed_password
- role
- is_active
- created_at

Financial Record Model

Typical fields:

- id
- amount

- type
- category
- date
- notes
- created_at

SQLite is used for persistence, satisfying the data persistence requirement from the assignment.

Testing

Basic tests are included in:

tests/test_app.py

Run tests

pytest

Or:

python -m pytest

These tests help verify that the main application behavior works correctly and align with the optional enhancement of unit/integration tests.

Local Usage Flow

Use this order when testing the project:

1. Run the server

uvicorn app.main:app --reload

2. Open docs

http://127.0.0.1:8000/docs

3. Call /seed

Create demo users.

4. Call /auth/login

Get JWT token.

5. Authorize in Swagger

Paste:

Bearer your_token

6. Test endpoints

- /users
- /records
- /dashboard/summary
- /dashboard/category-totals
- /dashboard/recent-activity
- /dashboard/monthly-trends

Assumptions Made

Since the assignment allows reasonable assumptions when something is not explicitly defined, this project uses the following assumptions:

- SQLite is sufficient for persistence in an internship assignment
- JWT authentication is used for simplicity and API friendliness
- Three main roles are enough to demonstrate access control
- Dashboard analytics are computed from stored financial records
- Local execution is acceptable even without public deployment
- Swagger serves as local API documentation

Trade-offs

SQLite instead of PostgreSQL

SQLite keeps setup simple and avoids external dependencies, but it is not ideal for high-scale production use.

Local-only deployment

The project is designed for local evaluation and demonstration rather than cloud deployment.

Simple RBAC model

The project uses a straightforward role-based access model instead of a more complex permission matrix, keeping the implementation readable and aligned with the assignment's evaluation criteria.

This project covers the core assignment requirements:

- User and role management
- Financial record creation, viewing, updating, deleting, and filtering
- Dashboard summary endpoints for totals, recent activity, category totals, and trends
- Backend-level role enforcement

- Validation and proper error handling
- Persistent storage using SQLite
- Optional enhancements like token auth, tests, and API documentation